

Práctica 1 - Implementación del Analizador Léxico para MiniLang

1. Introducción

En esta práctica se implementará el **analizador léxico (lexer)** del mini-lenguaje MiniLang, el lenguaje fuente que se utilizará durante el curso para construir un compilador de manera incremental.

El análisis léxico es la primera fase del proceso de compilación. Su función consiste en transformar el código fuente en una secuencia de tokens que representan las unidades léxicas del lenguaje. Por ejemplo:

Código

```
x = 10 + y;
```

Tokens

```
IDENT(x)
```

```
ASSIGN
```

```
NUM(10)
```

```
PLUS
```

```
IDENT(y)
```

```
SEMICOLON
```

2. Objetivos

Al finalizar esta práctica, el estudiante será capaz de:

- Comprender el papel del análisis léxico dentro de un compilador.
- Implementar un analizador léxico básico para un lenguaje de programación.
- Reconocer tokens a partir de un flujo de caracteres.
- Transformar un programa fuente en una secuencia estructurada de tokens.
- Manejar errores léxicos básicos.

3. Implementación

La elección del lenguaje de programación en el que deberá implementarse el analizador léxico es libre, pudiendo ser Java, Python, Go, o cualquier otro que sea más cómodo para cada estudiante.

En esta práctica se implementará un lexer manual, sin utilizar generadores automáticos de analizadores léxicos, lo cual permite comprender con mayor claridad cómo funciona el análisis léxico.

El analizador léxico debe:

1. Leer un archivo o cadena de texto que contenga un programa MiniLang.
2. Analizar el contenido carácter por carácter.
3. Identificar los tokens válidos.
4. Generar una lista de tokens en orden.
5. Ignorar espacios en blanco y saltos de línea.
6. Reportar errores léxicos si se encuentra un símbolo no reconocido.

Esto implica escribir lógica para reconocer identificadores, números y operadores, e ignorar espacios en blanco o comentarios. Ejemplo:

```
if (isLetter(c))  
leerIdentificador()
```

```
if (isDigit(c))  
leerNumero()
```

```
if (c == '+')  
token = PLUS
```

4. Ejemplos de entrada

Ejemplo de entrada válida

```
program ejemplo {  
  int x;  
  x = 5 + 3;  
  print(x);  
}
```

Salida esperada

```
(PROGRAM, program)  
(ID, ejemplo)  
(LBACE, {)  
(INT, int)  
(ID, x)  
(SEMICOLON, ;)  
(ID, x)  
(ASSIGN, =)
```

(NUM, 5)
(PLUS, +)
(NUM, 3)
(SEMICOLON, ;)
(PRINT, print)
(LPAREN, ()
(ID, x)
(RPAREN,)
(SEMICOLON, ;)
(RBRACE, }

Nota: El formato exacto de impresión puede variar, pero debe ser claro y consistente.

Ejemplo de entrada inválida

x = 5 @ 3;

Salida esperada

Error léxico: símbolo '@' no reconocido

5. Entregables

1. Código fuente del lexer.
2. Documento breve que explique:
 - a. Cómo funciona su implementación.
 - b. Cómo ejecutar el programa.
 - c. Descripción de las pruebas realizadas, incluyendo al menos 2 ejemplos de programas MiniLang y las respectivas salidas generadas por el lexer.

- **Ejercicio individual.**
- **Entrega:** Martes 29 de Septiembre (**antes de la clase**).
- Enviar entregables por correo electrónico.
- Presentación breve (del código en ejecución) en clase.

Rúbrica para evaluación de ejercicios prácticos y tareas

Calificación	Criterio
Excelente	1. Corrección técnica. El programa funciona correctamente para todos los casos requeridos y maneja errores adecuadamente. 2. Diseño y estructura del código. Código modular, claro, organizado, con funciones/clases bien definidas. 3. Correcta implementación del modelo teórico. Refleja correctamente la teoría vista en clase. 4. Manejo de casos de prueba y validación. Incluye casos de prueba adecuados y demuestra verificación. 5. Documentación y claridad. Comentarios adecuados, README (o documento PDF) claro, explicación breve del funcionamiento.
Bueno	1. Corrección técnica. Funciona en la mayoría de los casos, con fallos menores. 2. Diseño y estructura del código. Estructura adecuada con algunos problemas menores de organización. 3. Correcta implementación del modelo teórico. Existen pequeñas imprecisiones formales. 4. Manejo de casos de prueba y validación. Incluye algunos casos de prueba pero no son exhaustivos. 5. Documentación y claridad. Documentación suficiente pero mejorable.
Suficiente	1. Corrección técnica. Presenta errores funcionales pero muestra comprensión parcial. 2. Diseño y estructura del código. Código poco modular o con estructura confusa. 3. Correcta implementación del modelo teórico. Existen errores conceptuales leves. 4. Manejo de casos de prueba y validación. Se realizaron pruebas mínimas. 5. Documentación y claridad. Incluye comentarios mínimos.
Insuficiente	1. Corrección técnica. No funciona o no cumple con lo solicitado.

Calificación	Criterio
	2. Diseño y estructura del código. Código desorganizado o difícil de entender. 3. Correcta implementación del modelo teórico. Existen errores graves en la aplicación teórica. 4. Manejo de casos de prueba y validación. Sin pruebas o sin evidencia de validación. 5. Documentación y claridad. Sin documentación.